UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lecture 5: Learning with images - CNNs

What is a Convolutional Neural Network (CNN) and historically successful CNN architectures

Instructor: Thomas Lacombe

Learning outcomes

- **Explain** how convolutional neural networks (CNNs) and vision transformers (ViTs) process visual data by analysing their core components (e.g., convolutions, pooling, patch embeddings, positional encodings, and transformer encoder blocks).

- **Compute and apply** convolution, padding, stride, and pooling operations to transform an input image or feature map, and **predict** the resulting output shapes.

- **Compare and contrast** major CNN architectures (e.g., LeNet, AlexNet, VGG, Inception, ResNet) by identifying key design choices and **evaluating** their impact on accuracy, parameter count, compute cost, and training behaviour.

Disclaimer/Acknowledgment:

The following slides are reusing some of the content from the Stanford CS230 and CS231n class:

https://stanford.edu/~shervine/teaching/cs-230/

https://cs231n.github.io/convolutional-networks/

Some content is also reused from the CS230 Stanford course slides created by Andrew Ng, available at https://cs230.stanford.edu/syllabus/.

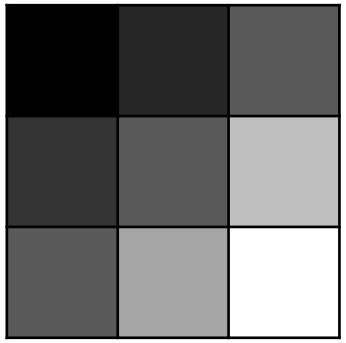
Extra resources

If you want to go further:

- *Hands-on Machine Learning with Scikit-Learn and PyTorch*, Géron A.
  - Chapter 12.
- *Deep Learning*, Goodfellow I. et al.
  - Part II Deep Networks, chap. 9.

Image representations

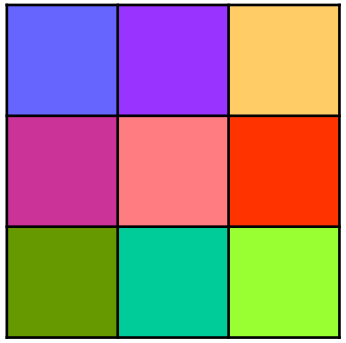
- Digital images: **structured grids of numerical values** (called *pixels*)
- A pixel encodes information about colour or intensity at a specific spatial location



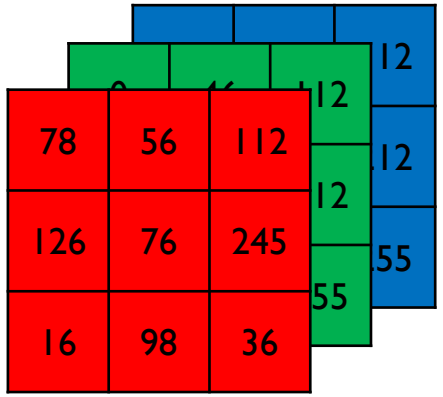
Grayscale →

0	46	112
56	157	212
146	198	255

H = 3 pixels
W = 3 pixels
C = 1 channel



RGB →



H = 3 pixels
W = 3 pixels
C = 3 channels

Image representations

- Pratical notes
  - In practice, pixel values typically **scaled to range [0, 1]** and **normalised per channel** over the dataset to improve numerical stability and training convergence.
  - In Deep Learning framework, images often stored as tensors of shape (C, H, W) or (H, W, C)
  - Images can be represented in **alternative colour spaces** (e.g., HSL/HSB/HIS, CMYK, CIE LAB, etc)
  - Some image modalities contain **more than 3 channels** (e.g., multispectral and hyperspectral images)

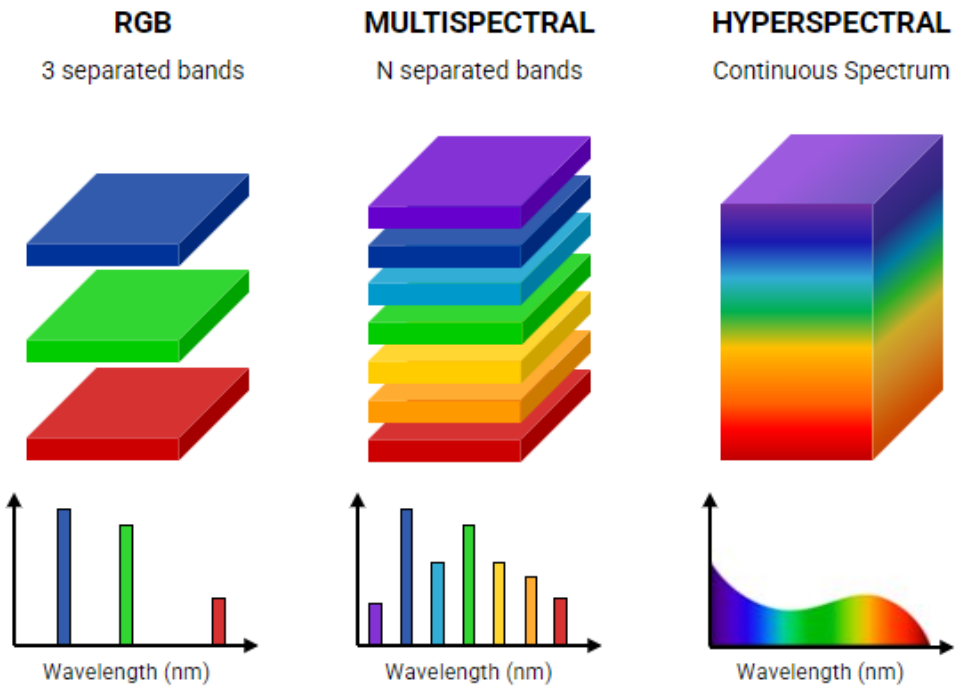
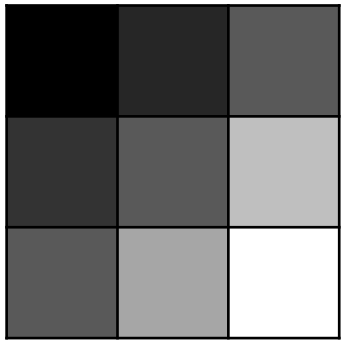


Image basic pre-processing



Grayscale digitisation →

0	46	112
56	157	212
146	198	255

Scaling to [0, 1] →

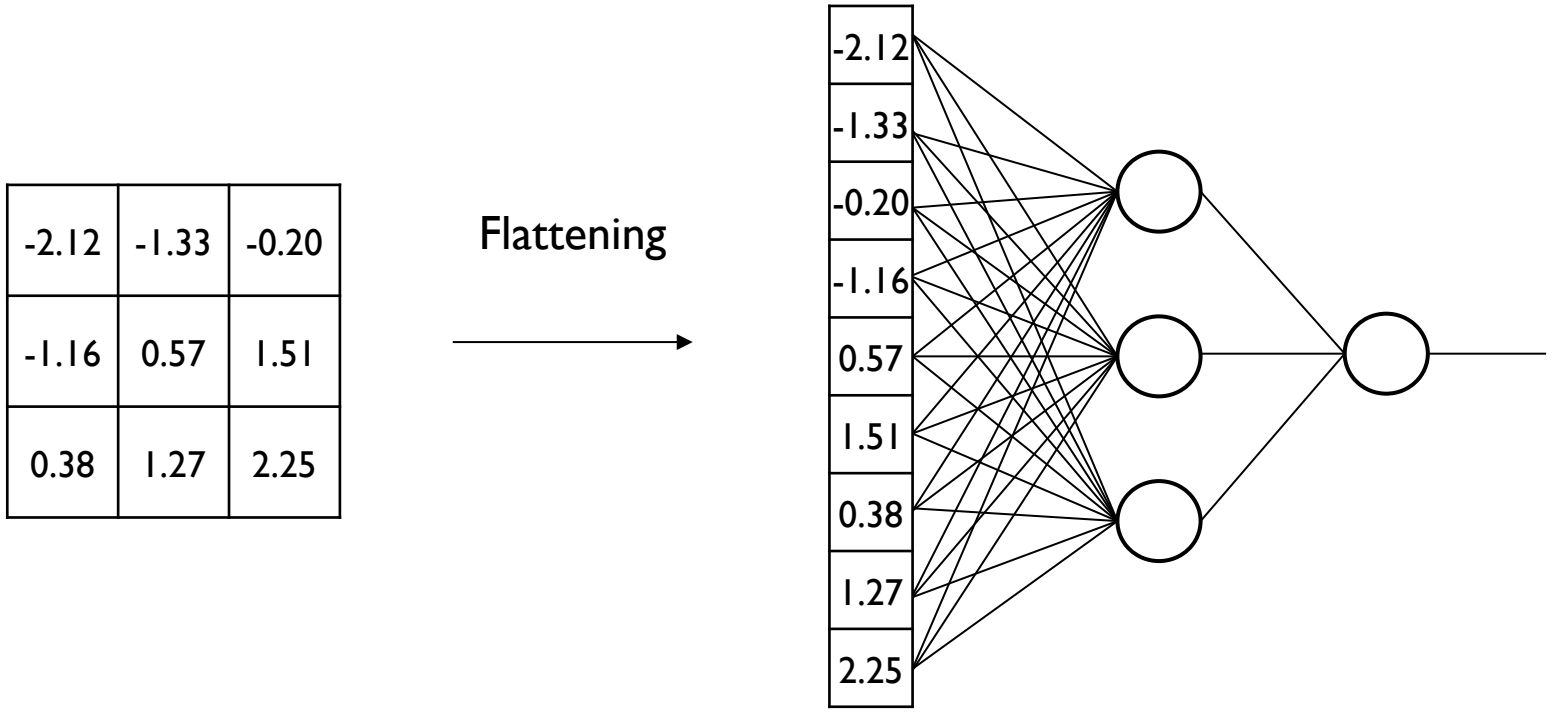
0.00	0.18	0.44
0.22	0.62	0.83
0.57	0.78	1.00

Standardise with dataset mean and std →

-2.12	-1.33	-0.20
-1.16	0.57	1.51
0.38	1.27	2.25

How to deal with image inputs?

- What about using classic feed forward ANN with flatten images?



↳ Ok for small images, but the number of parameters would go up quickly with large ones!

How to deal with image inputs?

- Could use classic feed forward ANN with flatten images.
  - But the number of parameters in the first hidden layer would go up quickly!

32 x 32 x 3 images → 3072 parameters for each neuron in the first hidden layer.
100 x 100 x 3 → 30 000 parameters per neuron.
200 x 200 x 3 images → 120 000 parameters per neuron.
2000 x 2000 x 3 images → 12M parameters per neuron.

- Can lead to overfitting.
- Does not perform well with complex images (e.g., spatial and temporal pixel dependencies).

Convolutional Neural Network (CNN)

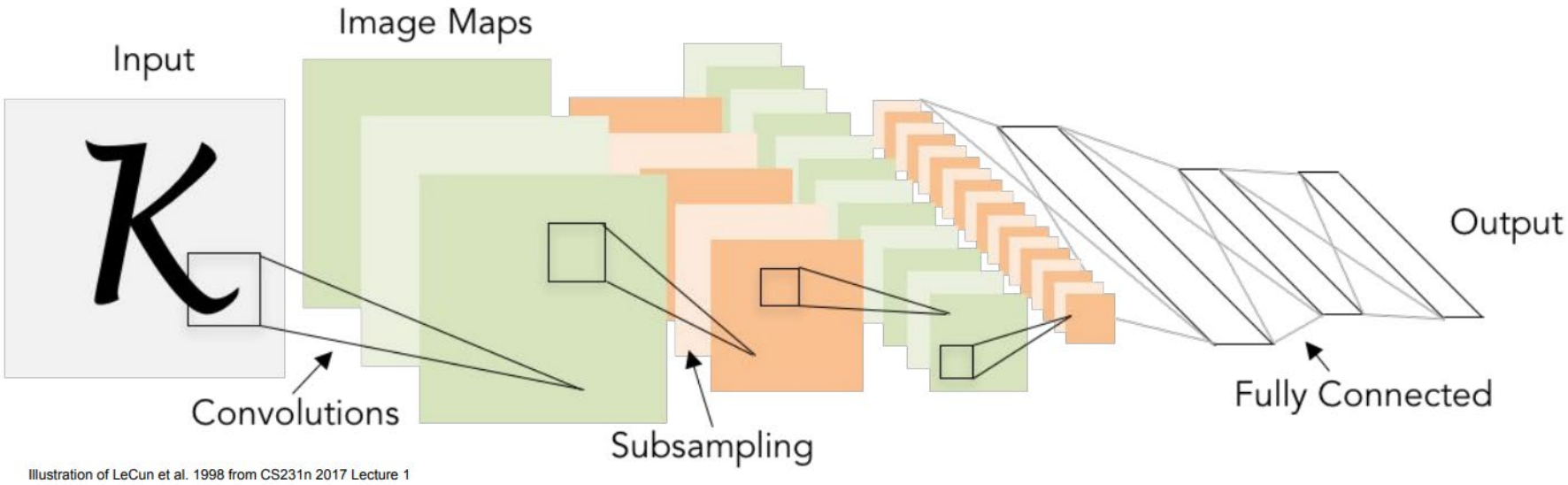


Illustration of LeCun et al. 1998 from CS231n 2017 Lecture 1

Main differences with a classic ANN:

- Explicit assumption that inputs are images.
- Introduce new types of layers: convolution and pooling layers.

Convolutional Neural Network (CNN)

- A CNN (or ConvNet) uses new types of layers to:
  - Make images easier to process.
  - Extract meaningful features from the image.

- Main novelty lies in the use of convolutions.

- Convolution already used a lot in image processing to filter images using **kernels** (e.g., edge detection).

- Today, CNNs are part of the most popular architectures for computer vision tasks, alongside with other Transformer-based architectures (e.g., Vision Transformer).

Where do convolutions come from?

- Filters in computer vision/image processing.
- E.g., edge detection:

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

6 x 6

*

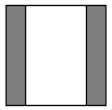
1	0	-1
1	0	-1
1	0	-1

3 x 3

=

-0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

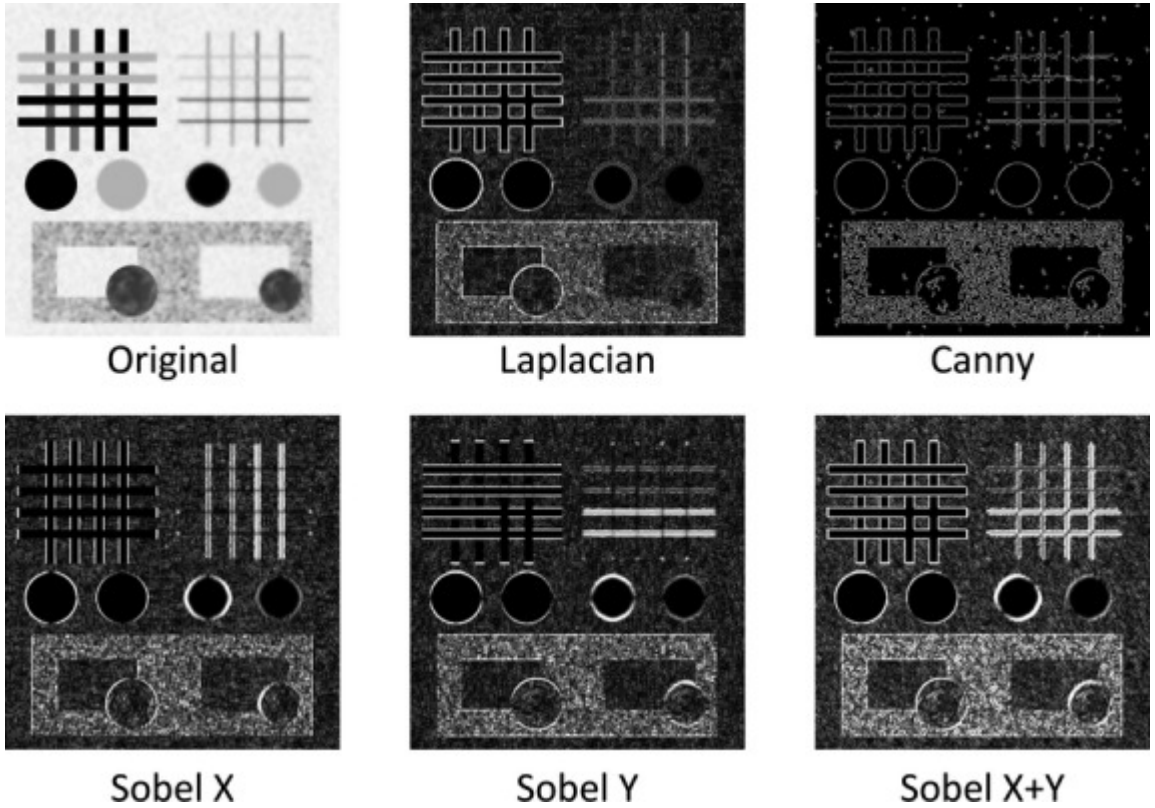
4 x 4



Andrew Ng

Edge detection

- Filtering examples:



Sobel

-1	0	1
-2	0	2
-1	0	1

Sobel X

1	2	1
0	0	0
-1	-2	-1

Sobel Y

Laplacian

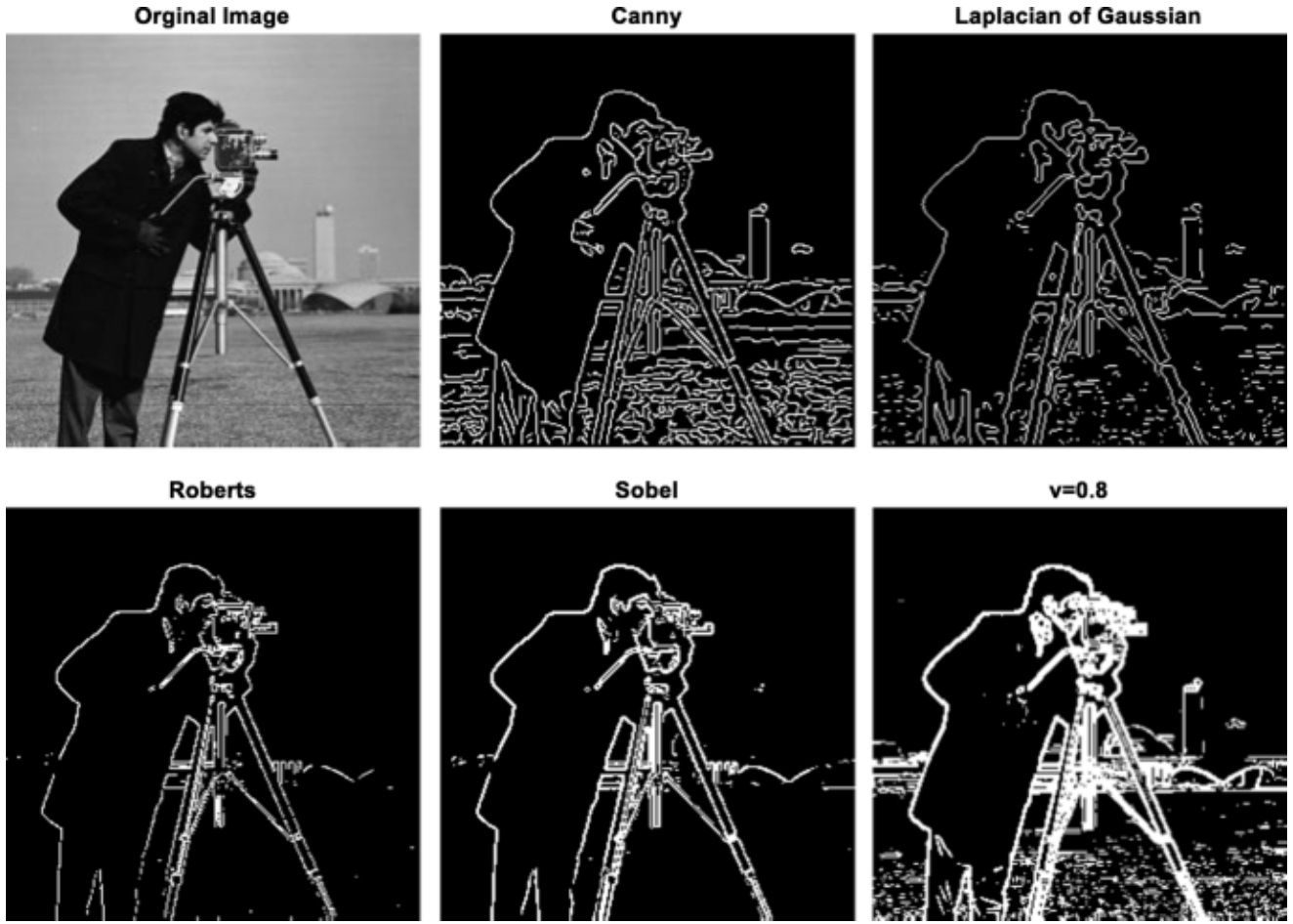
0	-1	0
-1	4	-1
0	-1	0

-1	-1	-1
-1	8	-1
-1	-1	-1

Image: Uchida, S. (2013). Image processing and recognition for biological images.

Edge detection

- Another example:



CNN learns the filter!

- What about learning the filters needed to extract features adapted to a target task?

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

$*$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$=$

Andrew Ng

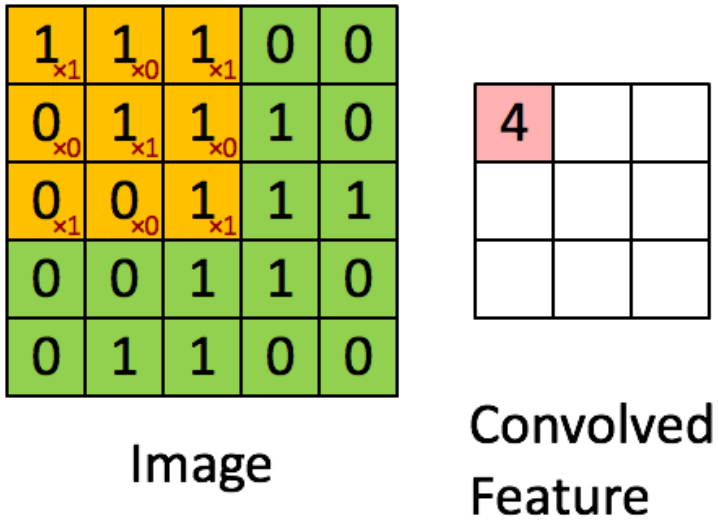
- Can learn values of the filter with backpropagation!

Convolutional layer

- Convolutional layer
  - Sliding kernel (filter) over the image.
  - Matrix multiplications + sum.

Convolution of a $f \times f$ filter with a $n \times n$ image:

- $n - f + 1 \times n - f + 1$ feature map (or activation map, or convolved feature).



Disadvantages:

- Shrinking size.
- Corner and border pixels don't contribute as much to the feature map (loosing information).

Gif source: https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Padding

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

padding p = 1

*

1	0	1
0	1	0
1	0	1

=

2	2	3	1	1
1	4	3	4	2
1	2	4	3	3
1	2	3	4	1
0	2	2	1	1

Convolution of a $f \times f$ filter with a $n \times n$ image, with padding (p pixel(s)):

- $n + 2p - f + 1 \times n + 2p - f + 1$ feature map.

Valid and same padding

- Valid padding = no padding.
  - Output: $n - f + 1 \times n - f + 1$ feature map.

- Same padding = padding to make output feature map the same size as the input.
  - Output: $n + 2p - f + 1 \times n + 2p - f + 1$ *feature map*.

What p should you use?

- $n + 2p - f + 1 = n$ → p = (f-1) / 2 (depends on filter/kernel size!)

→ Filter/ kernel size usually odd (3×3, 5×5, 7×7, …) to have an integer value for p when using same padding.

Strided convolution

- Convolution on very big images can be computationally expensive.
- Strided convolution can help reduce the computational cost.

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 2 & 4 \end{bmatrix}$$

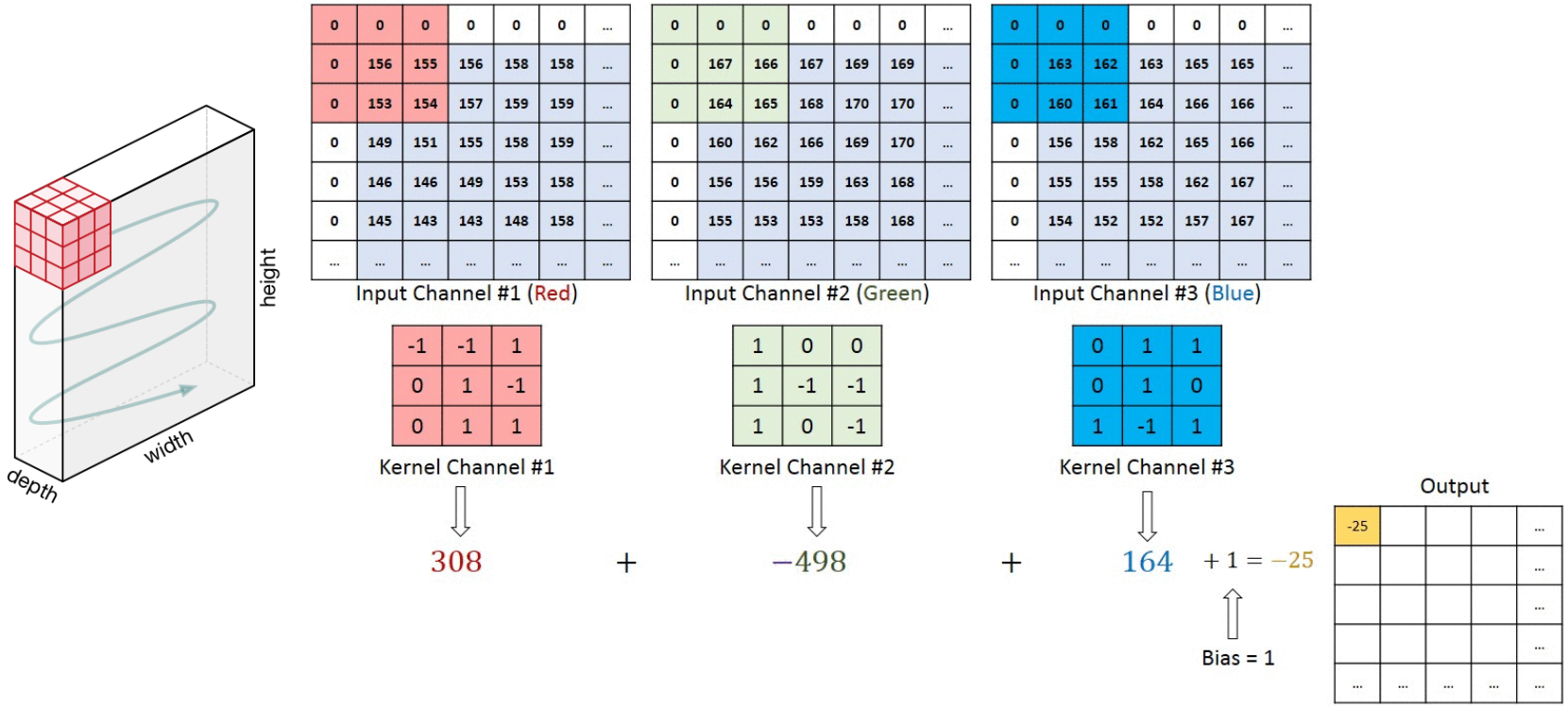
stride s = 2

Convolution of a $f \times f$ filter with a $n \times n$ image, with padding p and stride s:

$\frac{n+2p-f}{s}+1 \times \frac{n+2p-f}{s}+1$ feature map.

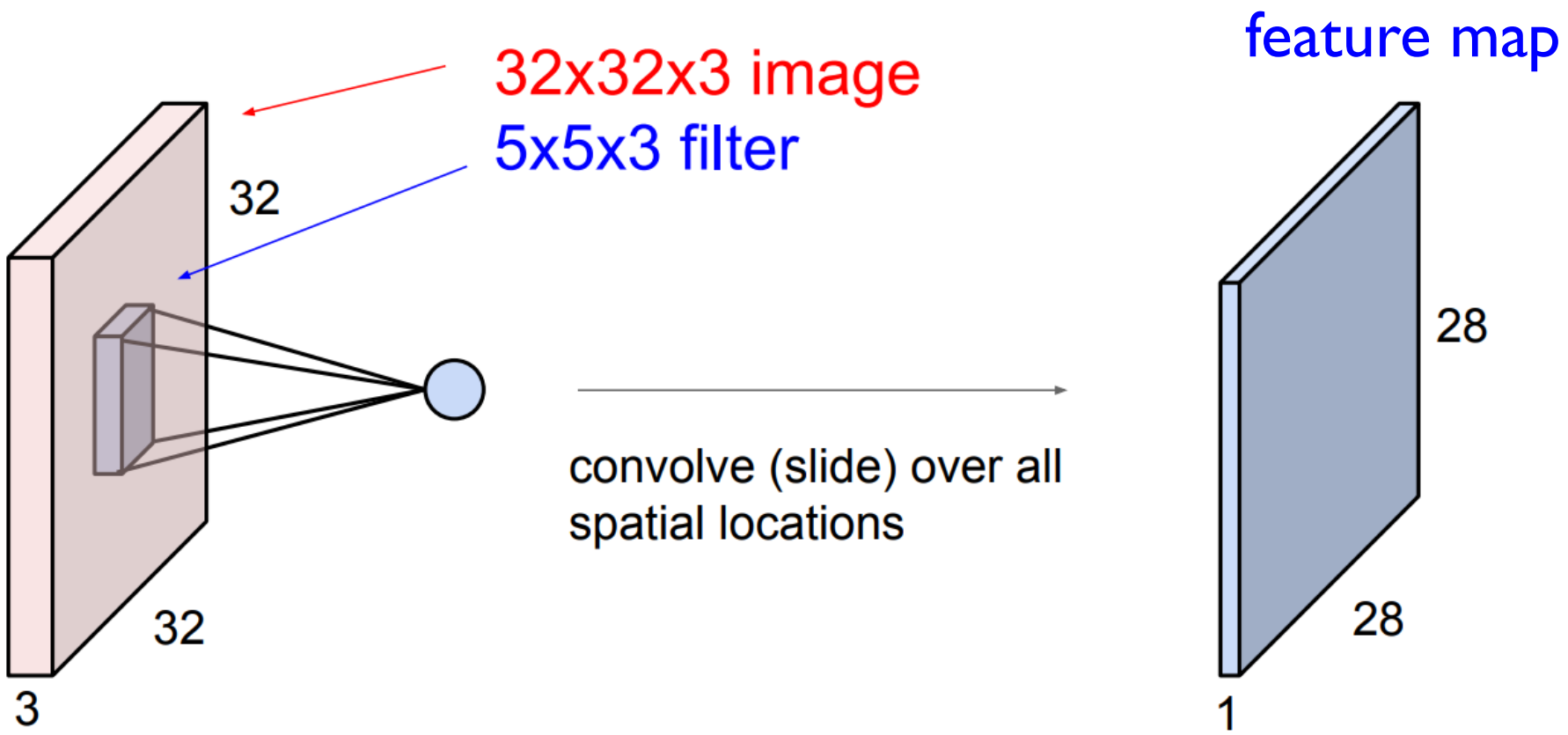
Images as volumes

Convolutions with volumes:



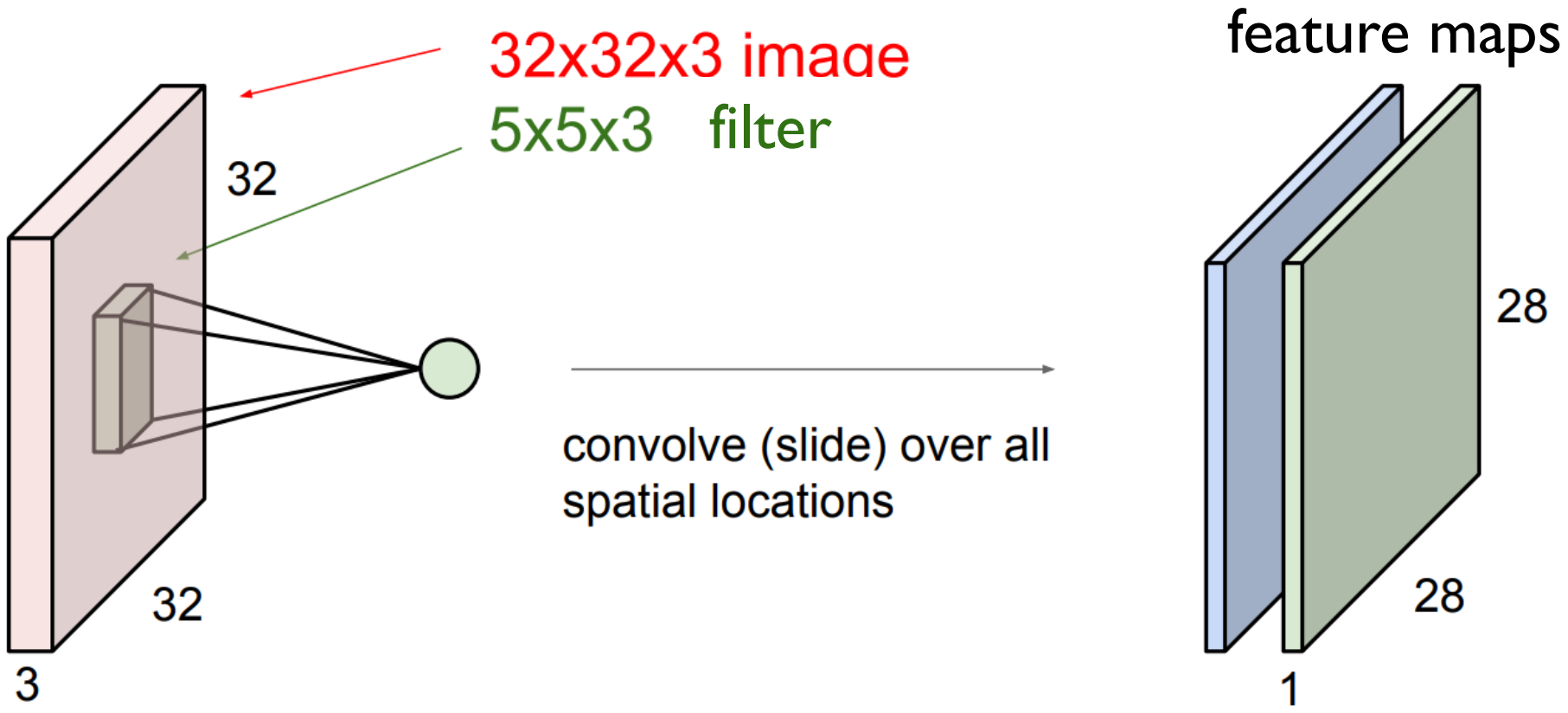
Gif source: https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Stacking kernels



Stanford CS231n

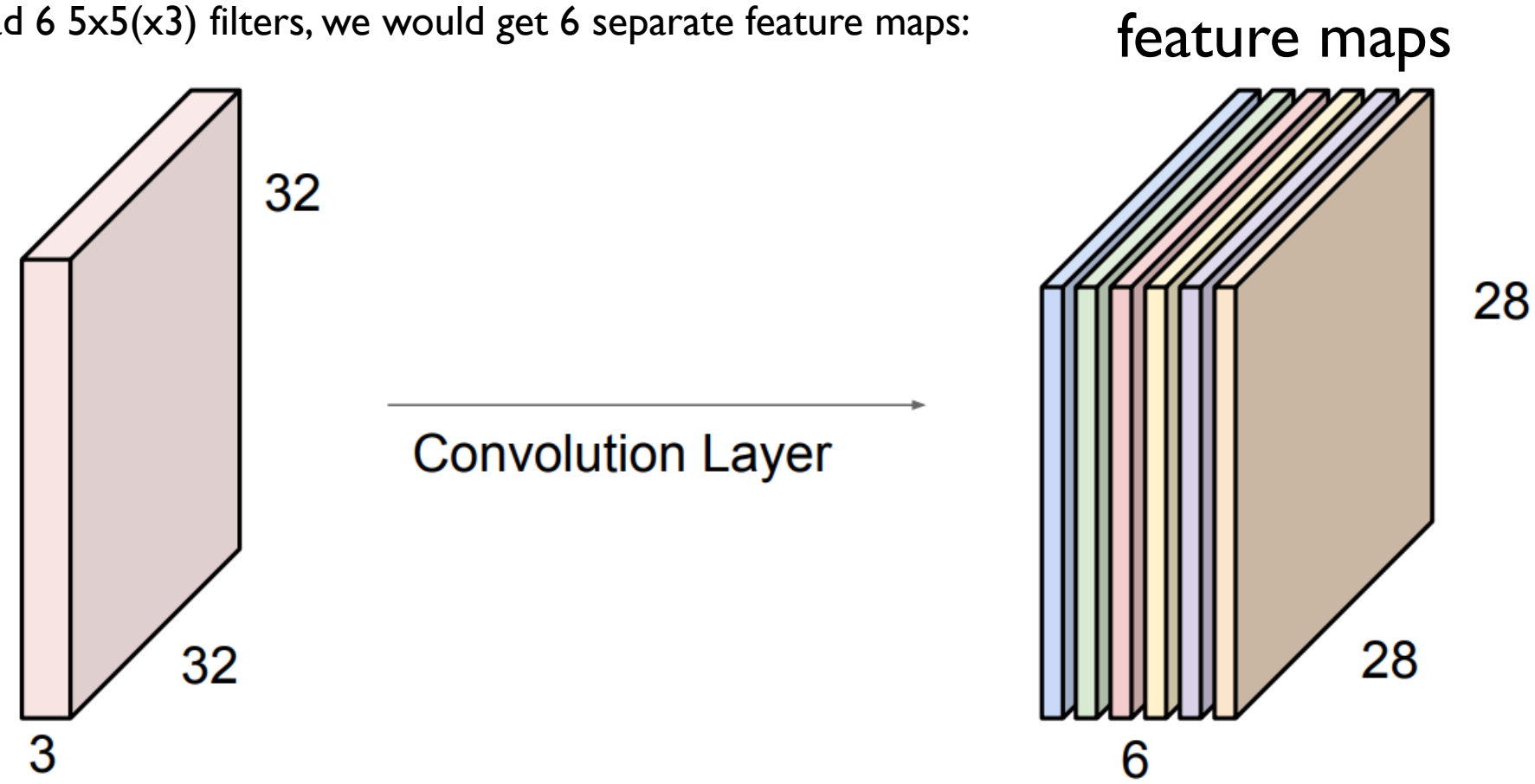
Stacking kernels



Stanford CS231n

Stacking kernels

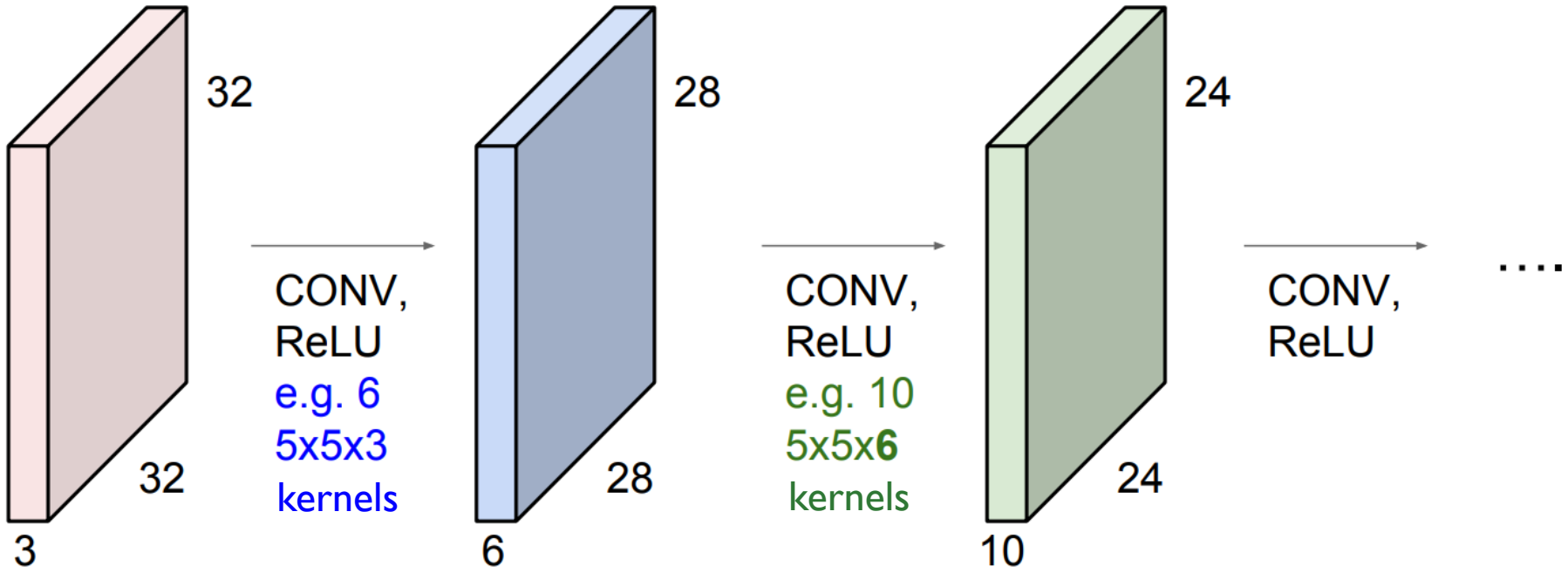
If we had 6 5x5(x3) filters, we would get 6 separate feature maps:



We stack these up to get a "new image" of size 28x28x6!

Stanford CS231n

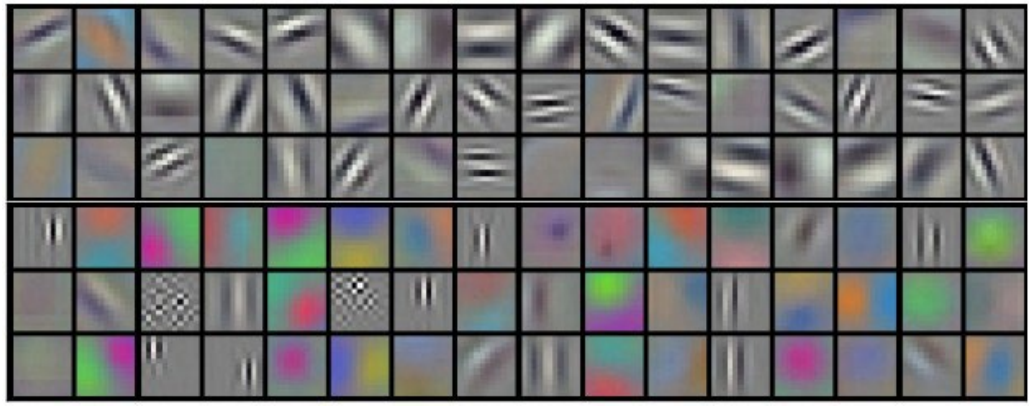
Stacking convolutional layers



Stanford CS231n

Parameter sharing

- Filter parameters are shared across the input.
- Total number of parameters only dependent on filter size and number, not input size!
- Drastically reduces the number of parameters in the network!
- Makes sense to share features across input:



AlexNet first layer 96 learned filter [Krizhevsky et al. 2012]

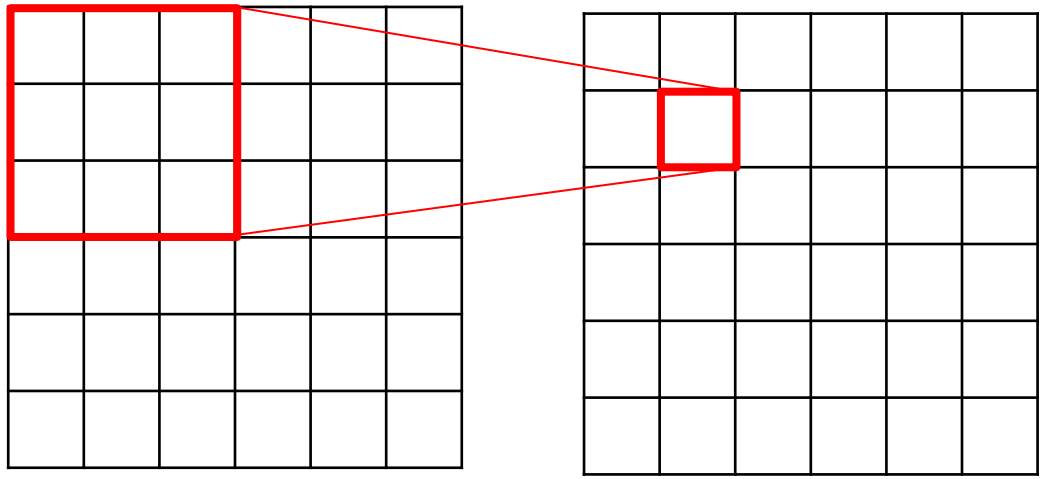
Receptive field

- **Receptive field** = region of the input influencing a neuron (≡ filter size)

- **Stacking convolutional layers increases receptive field w.r.t. input image**
  - Each layer aggregates information from a larger area
  - Deeper neurons “see” more of the image

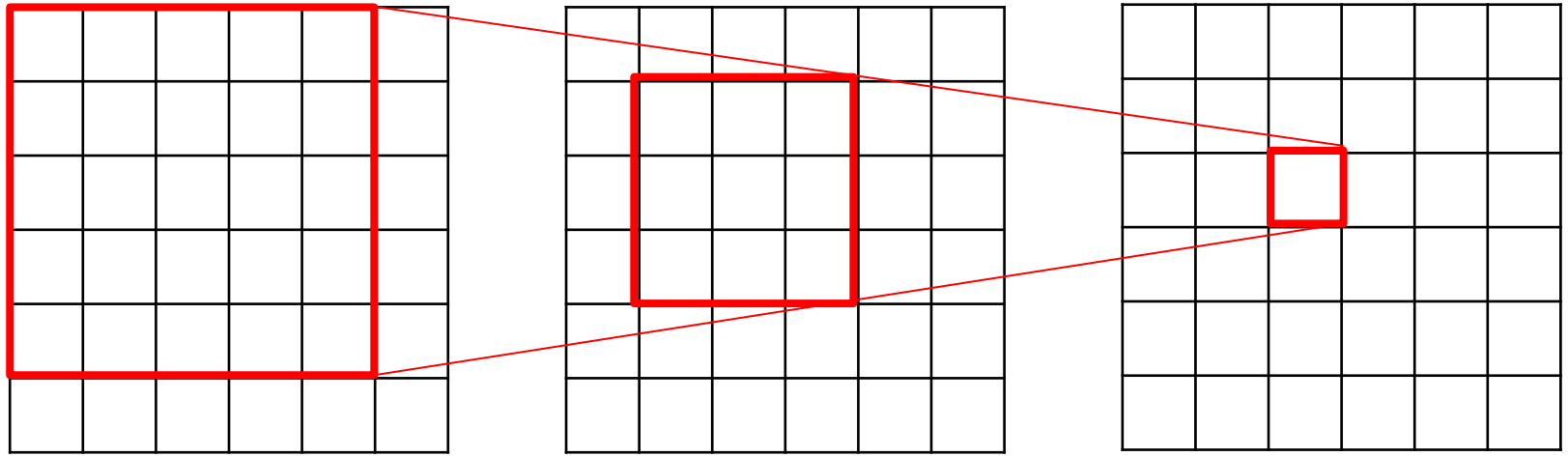
- **Hierarchical feature learning**
  - Early layers → local patterns (edges, textures)
  - Deeper layers → global structures (objects, context)

Receptive field



Feature map after one convolution layer with [3x2] filter (no padding, stride = 1) → local features

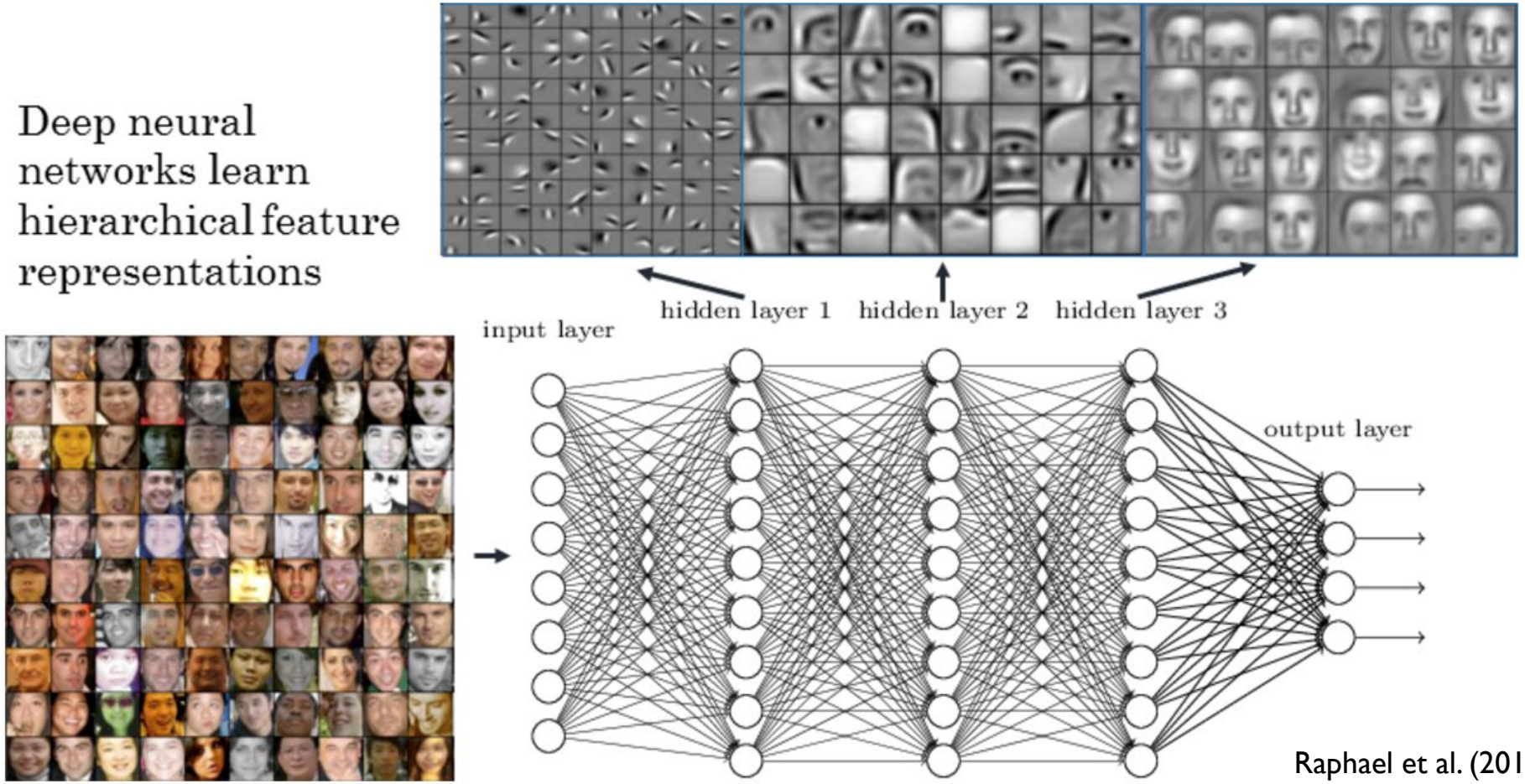
Input image | Conv1 | Feature map 1 | Conv2 | Feature map 2



Feature map after two stacked convolution layers with [3x3] filters → more global features

Hierarchical feature learning

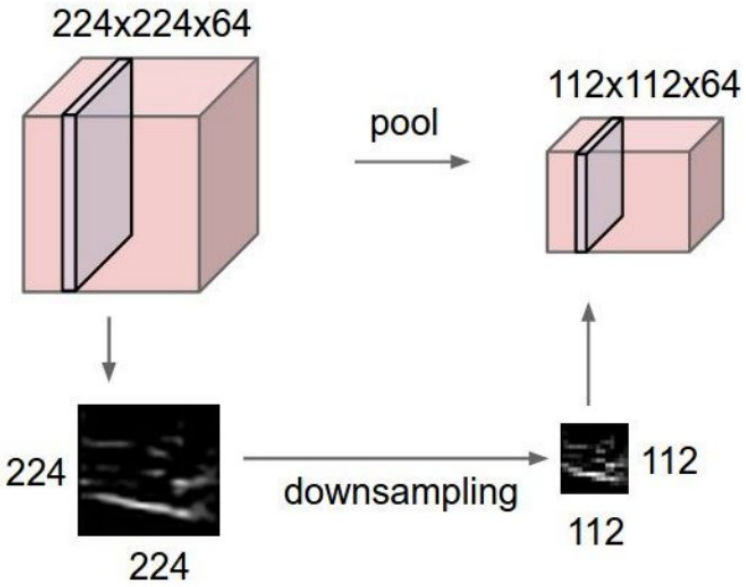
Deep neural networks learn hierarchical feature representations



Raphael et al. (2019), adapted from Honglak et al. (2011)

Pooling layer

- Pooling layer:
  - makes the representations smaller and more computationally manageable (dimensionality reduction).
  - reduces number of parameters, hence helps reduce overfitting and increase generalisation (extract important information).
  - operates over each feature map independently:



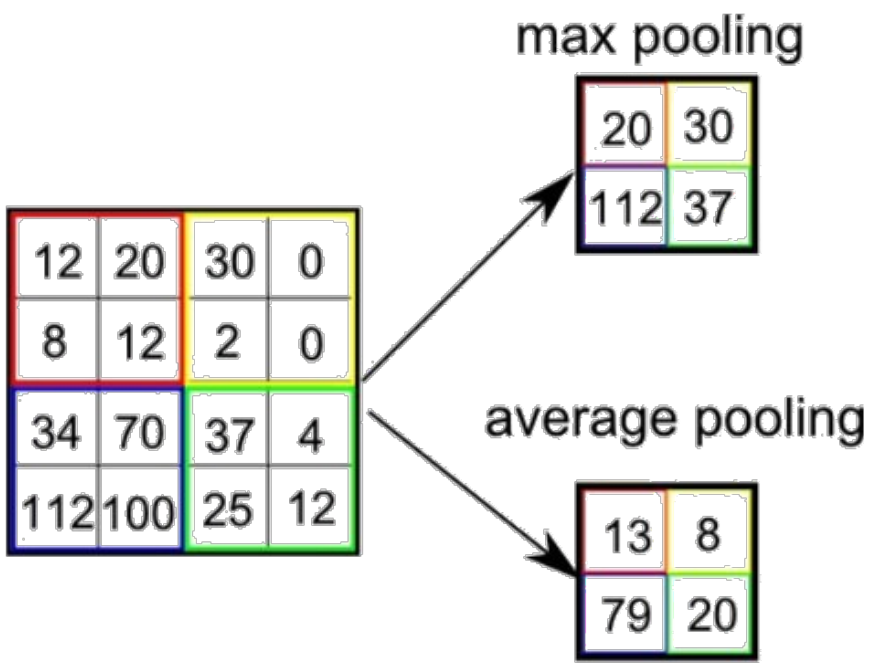
Stanford CS231n

Max pooling vs average pooling

- Two main types of pooling:

pool size f = 2

stride s = 2



- Max pooling preferred in practice, but choice depends on the task.

https://medium.com/@bdhuma/which-pooling-method-is-better-maxpooling-vs-minpooling-vs-average-pooling-95fb03f45a9

Image source: https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Hyperparameters

- Convolutional layer

  f : kernel size

  s : stride

  p : padding

  n'_C : number of filters

If input size is $n_H \times n_W \times n_C$:

Output size of convolution will be:

$$\left\lfloor \frac{n_H+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W+2p-f}{s} + 1 \right\rfloor \times n'_C$$

- Pooling layer

  f : kernel size

  s : stride

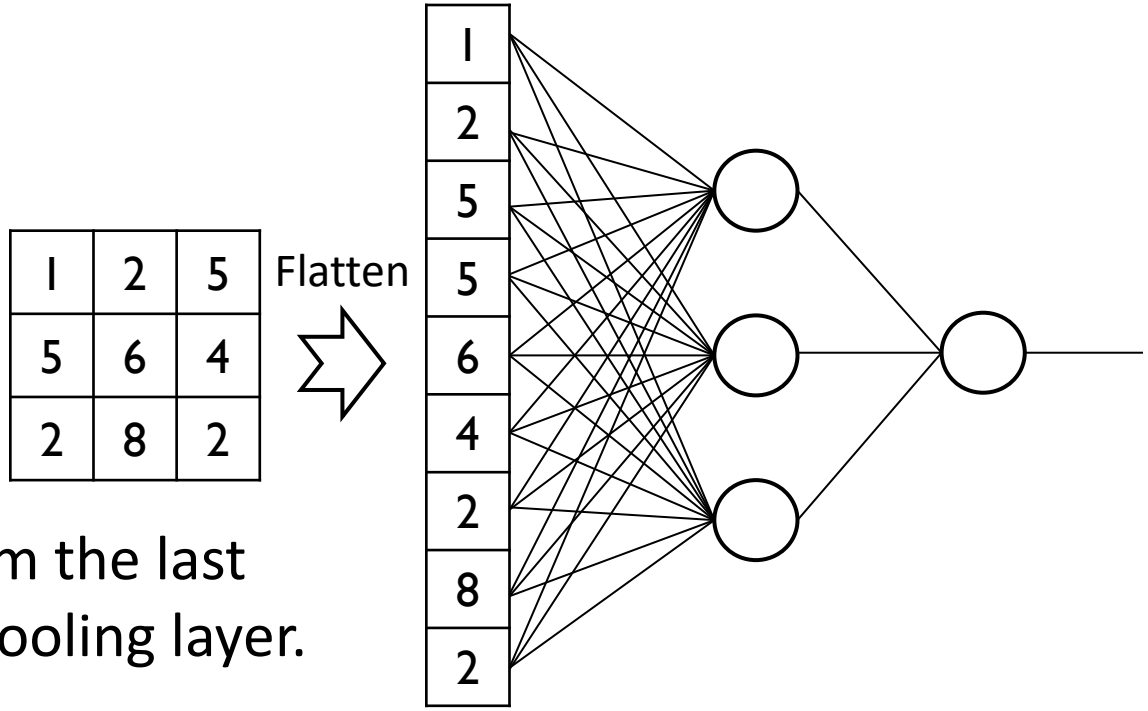
  Max or average pooling

Output size of pooling will be:

$$\left\lfloor \frac{n_H-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W-f}{s} + 1 \right\rfloor \times n_C$$

Fully connected layer

- Fully connected layer:
  - performs the classification.



Feature map from the last convolution or pooling layer.

Bringing everything together

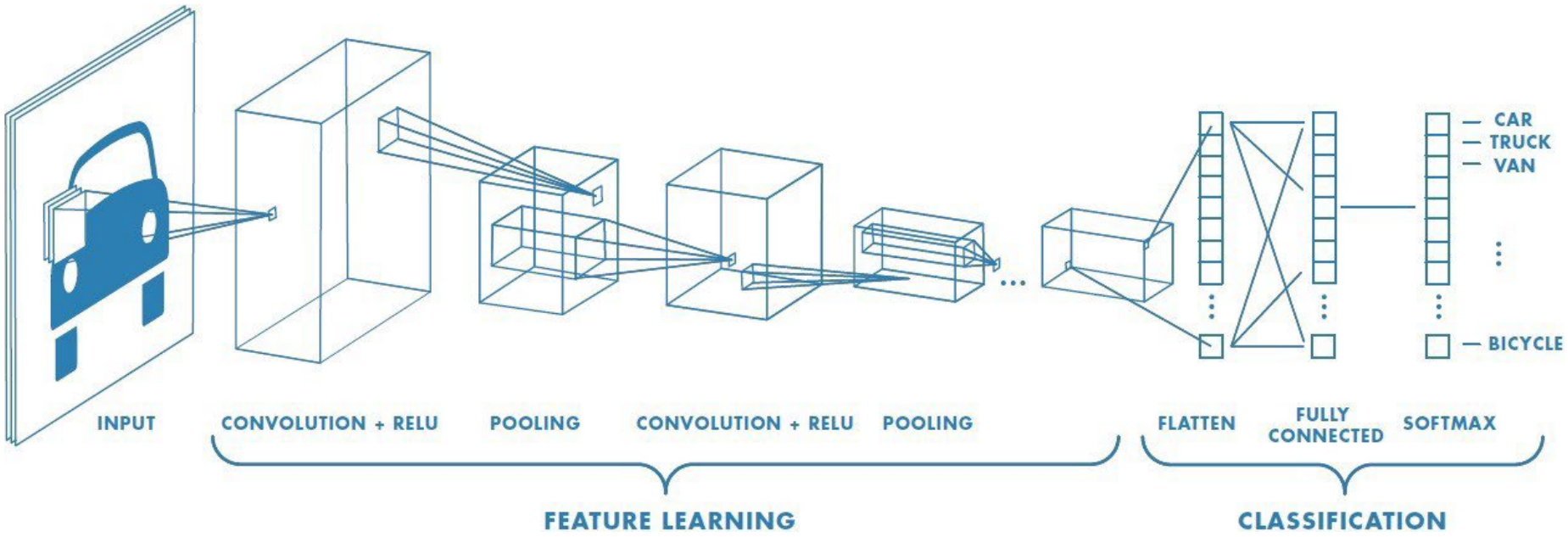


Image source: https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

ConvNet sizing

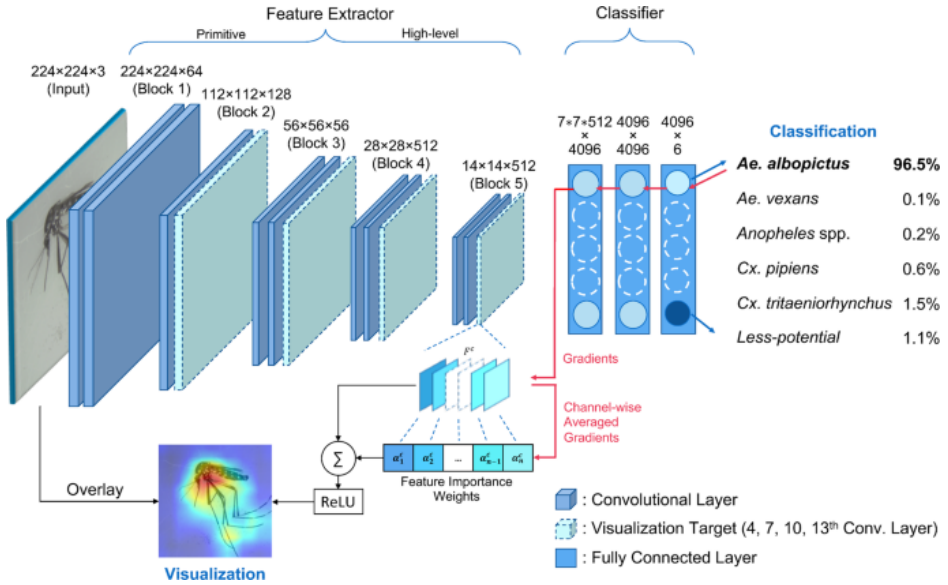
- Sizing a ConvNet can be tricky!
  - If input size and layer hyperparameters do not align → can end up with non-integer feature map sizes:
    - E.g., input [224x224x3] and [11x11x3] filter with p = 0 and s = 4 → (224-11)/4 + 1 = 54.25
    - But [224x224x3] and [11x11x3] filter with p = 3 and s = 4 → (224-11+3)/4 + 1 = 55

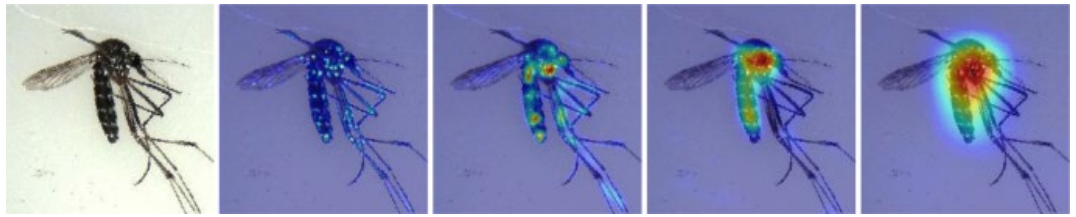
- Rules of thumb:

Rule	Rational
Choose input sizes divisible by powers of 2 (e.g., 32, 64, 128, 224, 384, 512)	Ensures consistent downsampling through multiple pooling/stride-2 layers without fractional feature maps
Use small convolutional kernels (typically 3×3, occasionally up to 5×5) with **stride = 1** and **same padding**	Preserves spatial resolution while increasing depth + improves parameter efficiency and enables deeper architectures
Let pooling handle spatial downsampling (commonly 2×2 with stride 2)	Provides controlled, progressive reduction of spatial dimensions without losing too much information + aligns naturally with power-of-2 input sizes

Adapted from: https://cs231n.github.io/convolutional-networks/

Visualising CNN predictions





The visualization of discriminative regions in the 4-, 7-, 10- and 13-th convolution layers of VGG-16 when an *Aedes albopictus* input image is given.

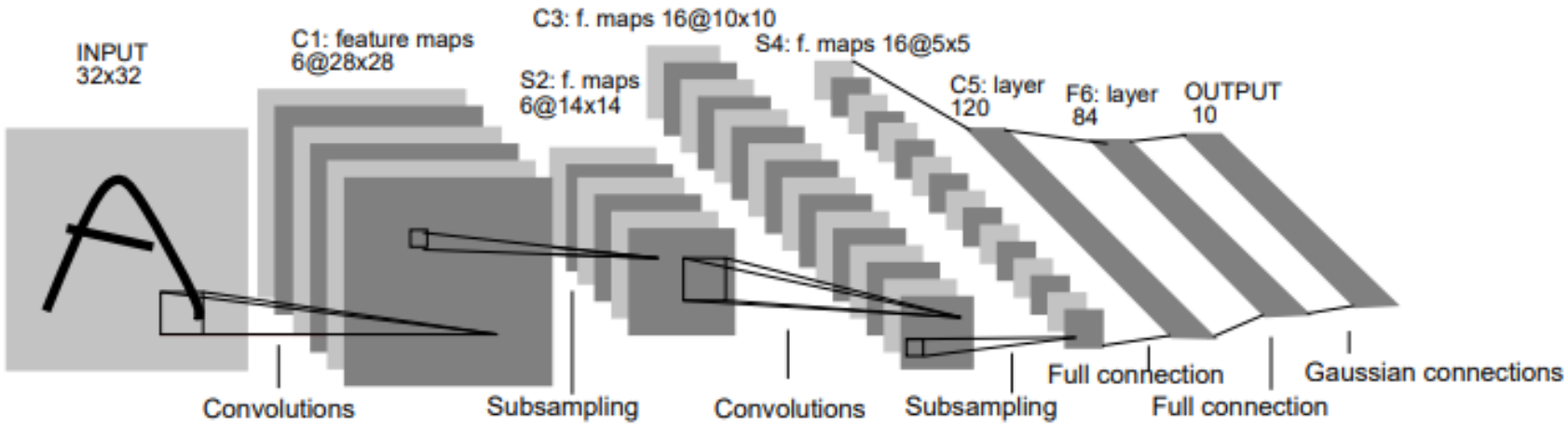
[Classification and Morphological Analysis of Vector Mosquitoes using Deep Convolutional Neural Networks, Park et al. 2020, Nature Scientific Reports]
https://www.nature.com/articles/s41598-020-57875-1.pdf
https://medium.com/@ODSC/visualizing-your-convolutional-neural-network-predictions-with-saliency-maps-9604eb03d766

A few successful CNN architectures

- CNN design is all about how to place the previous building blocks together.
- Good to see what works (case studies).
- Often, existing architectures can be reused on new tasks.
- LeNet, AlexNet, VGGNet, Inception/GoogleNet, ResNet

CNN case studies

- LeNet-5 1998



- Convolution kernels: kernel size f = 5 (5x5), stride s =1
- Pooling (subsampling) layers: kernel size f = 2 (2x2), stride s = 2
- Architecture: [CONV-POOL-CONV-POOL-FC-FC]

[LeCun 1998]
Stanford CS231n

CNN case studies

- AlexNet 2012

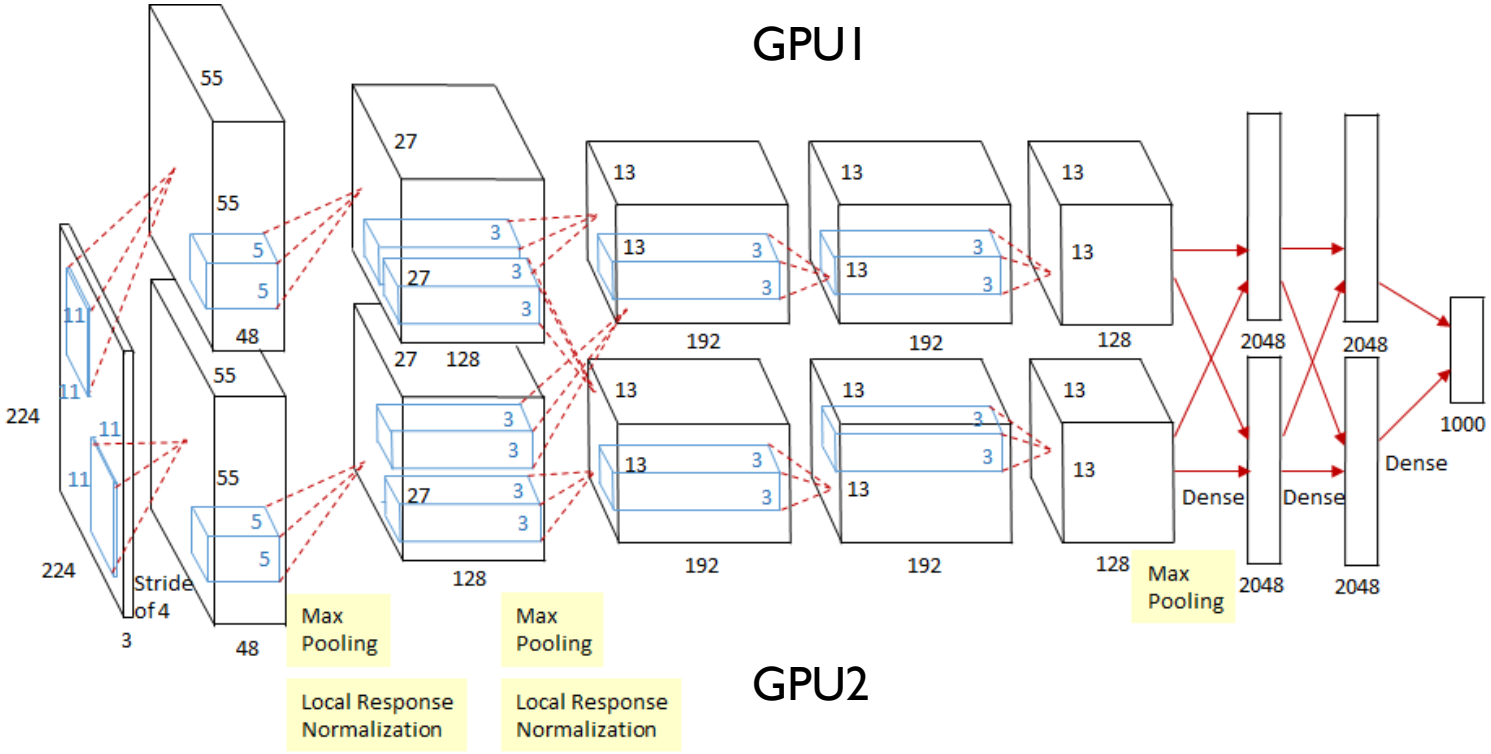


Image source: https://medium.com/coinmonks/paper-review-of-alexnet-caffenet-winner-in-ilsvrc-2012-image-classification-b93598314160

AlexNet paper :[Krizhevsky et al. 2012]

CNN case studies

- AlexNet

Full (simplified) AlexNet architecture:
[227x227x3] INPUT
[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0
[27x27x96] MAX POOL1: 3x3 filters at stride 2
[27x27x96] NORM1: Normalization layer
[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2
[13x13x256] MAX POOL2: 3x3 filters at stride 2
[13x13x256] NORM2: Normalization layer
[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1
[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1
[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1
[6x6x256] MAX POOL3: 3x3 filters at stride 2
[4096] FC6: 4096 neurons
[4096] FC7: 4096 neurons
[1000] FC8: 1000 neurons (class scores)

→ Each layer, except the last one, are split between the 2 GPUs.

Remarks/Hyperparameters:
- First use of ReLU
- Use of specific normalisation layers (obsolete now vs BN and modern architectures)
- Heavy data augmentation (x 2048!)
- Dropout: 0.5
- Batch size: 128
- SGD Momentum: 0.9
- Learning rate: 1e-2, reduced by 10 manually when val accuracy plateaus
- L2 regularisation weight decay: 5e-4

Stanford CS231n

CNN case studies

- VGGNet 2014

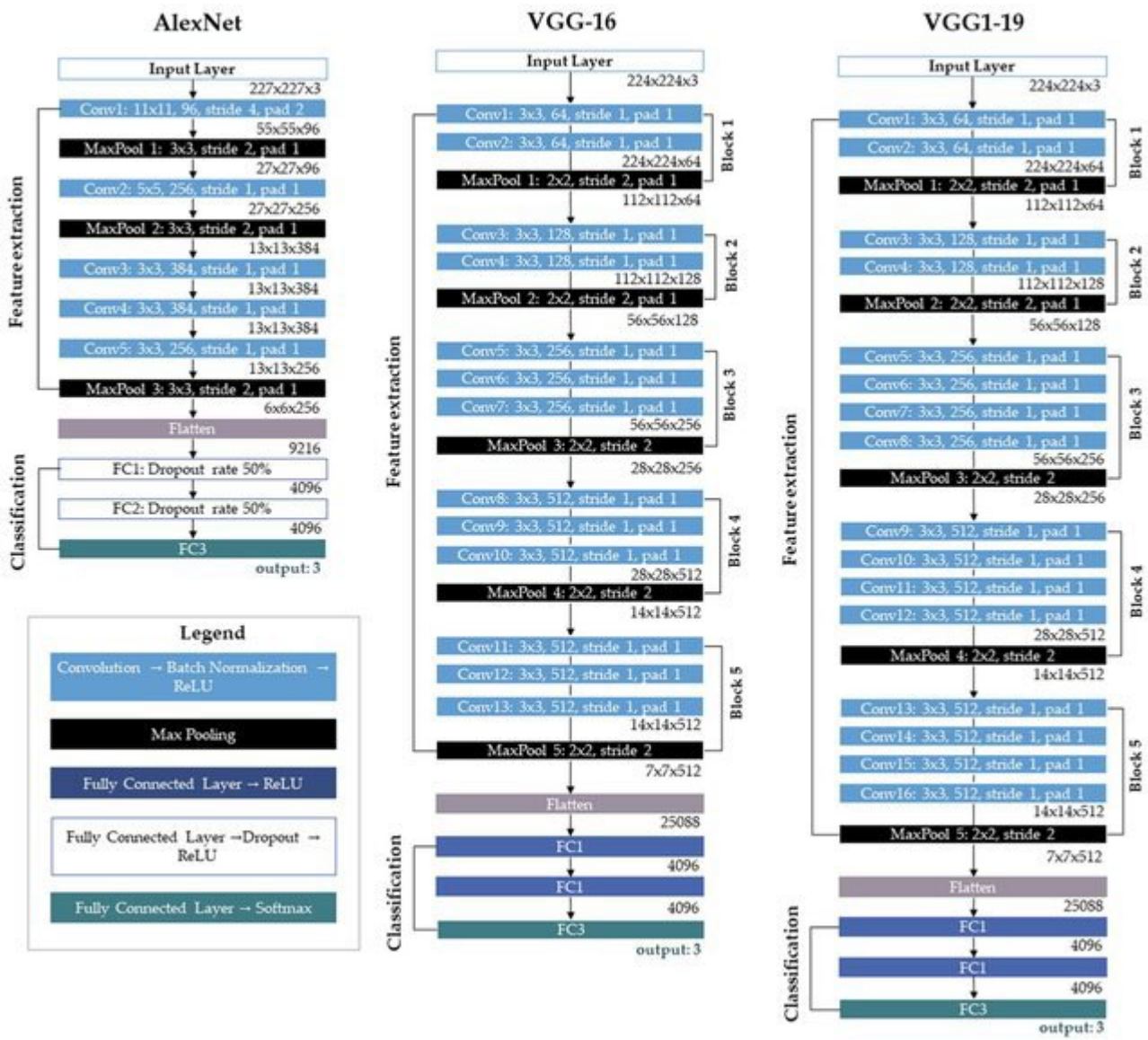
Small filters, Deeper networks

- Why small filters?

  Stack of small filters have the same effect than one larger one.

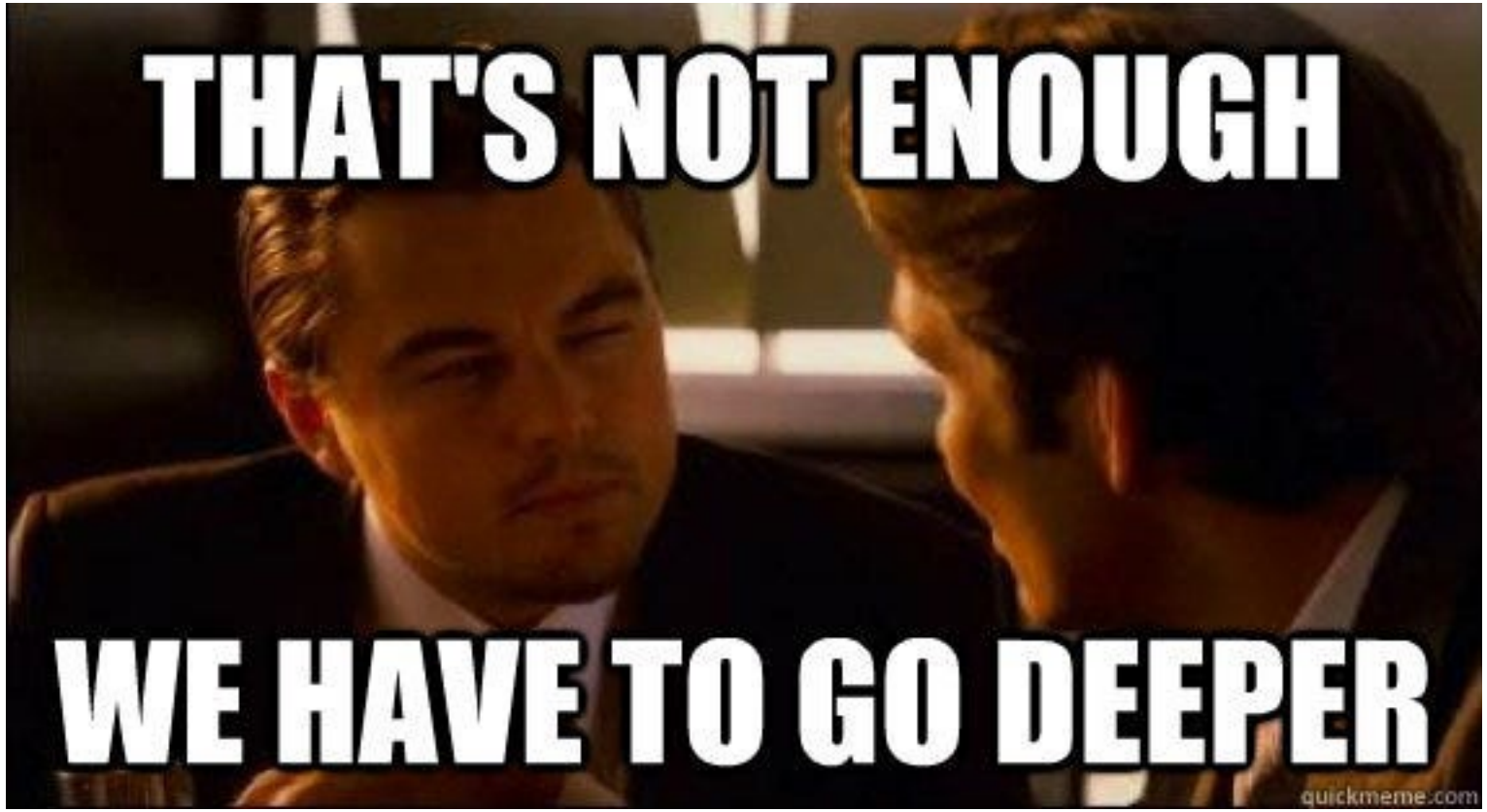
- Why deeper networks?

  More layers = more non-linearities.



VGG paper [Simonyan and Zisserman, 2014]
Image source: [Manataki et al. 2023]

CNN case studies

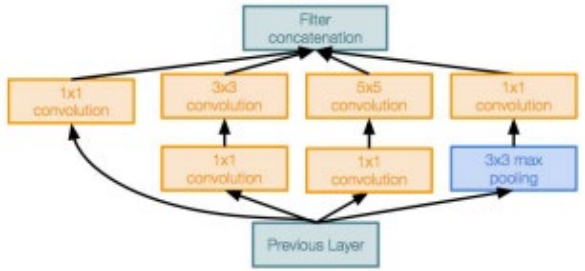


CNN case studies

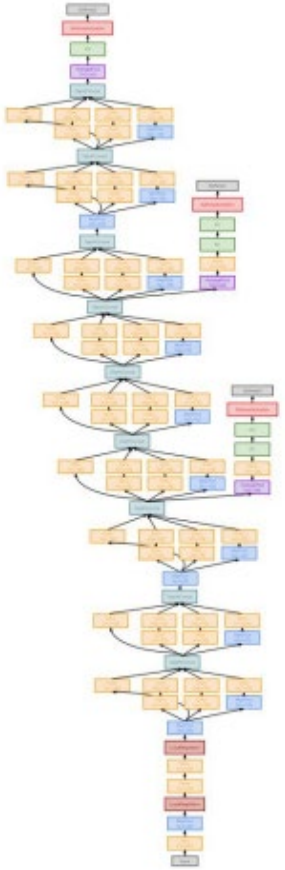
- GoogLeNet 2014

Deeper networks, with computational efficiency.

- 22 layers.
- Stack of Inception modules.
- Inception module = good and efficient local architecture, “network in the network”.
- 5 million parameters, 12x less than AlexNet.
- Auxiliary classification output branches to increase gradient signal at lower level.



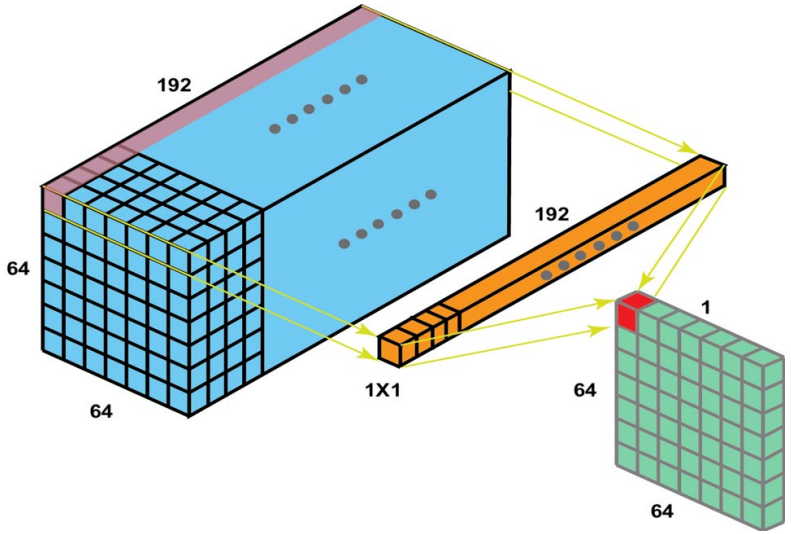
Inception module



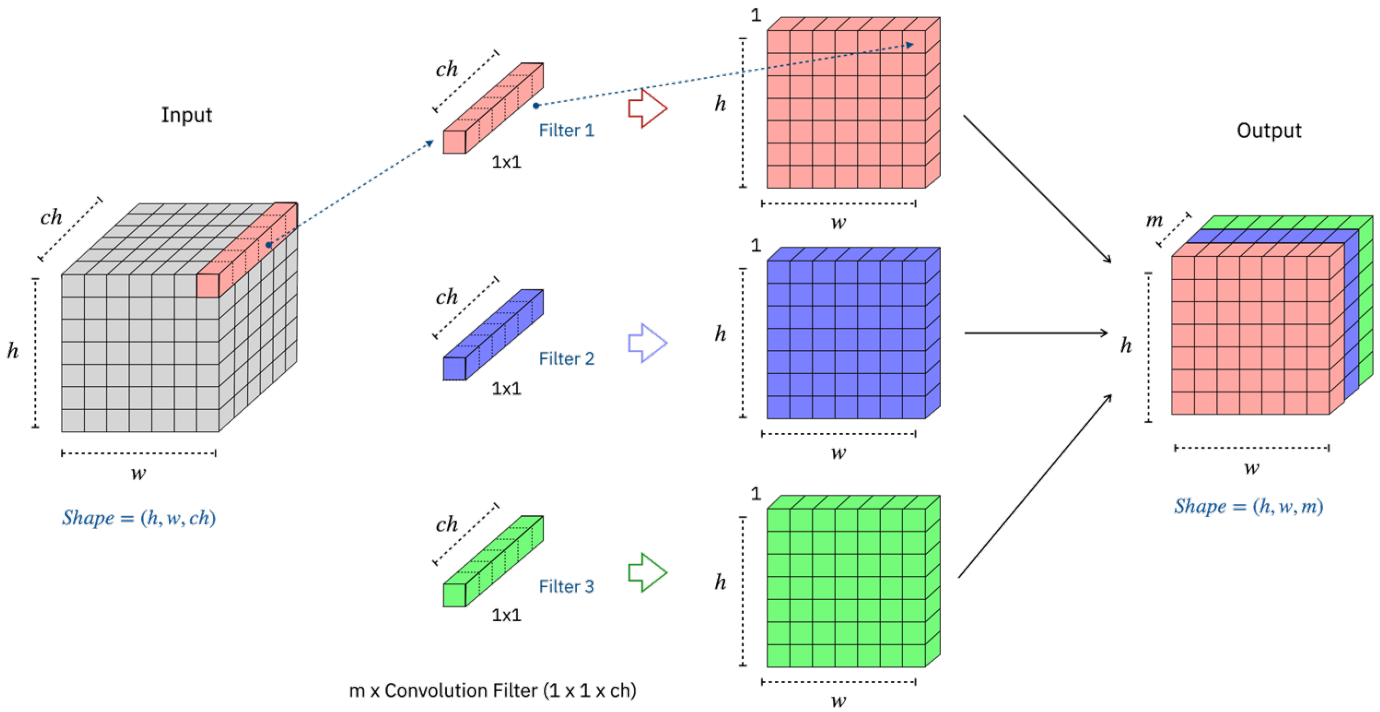
[Szegedy et al., 2014]
Stanford CS231n

1x1 convolution

- One 1x1 filter



- Several 1x1 filters



CNN case studies

- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners.

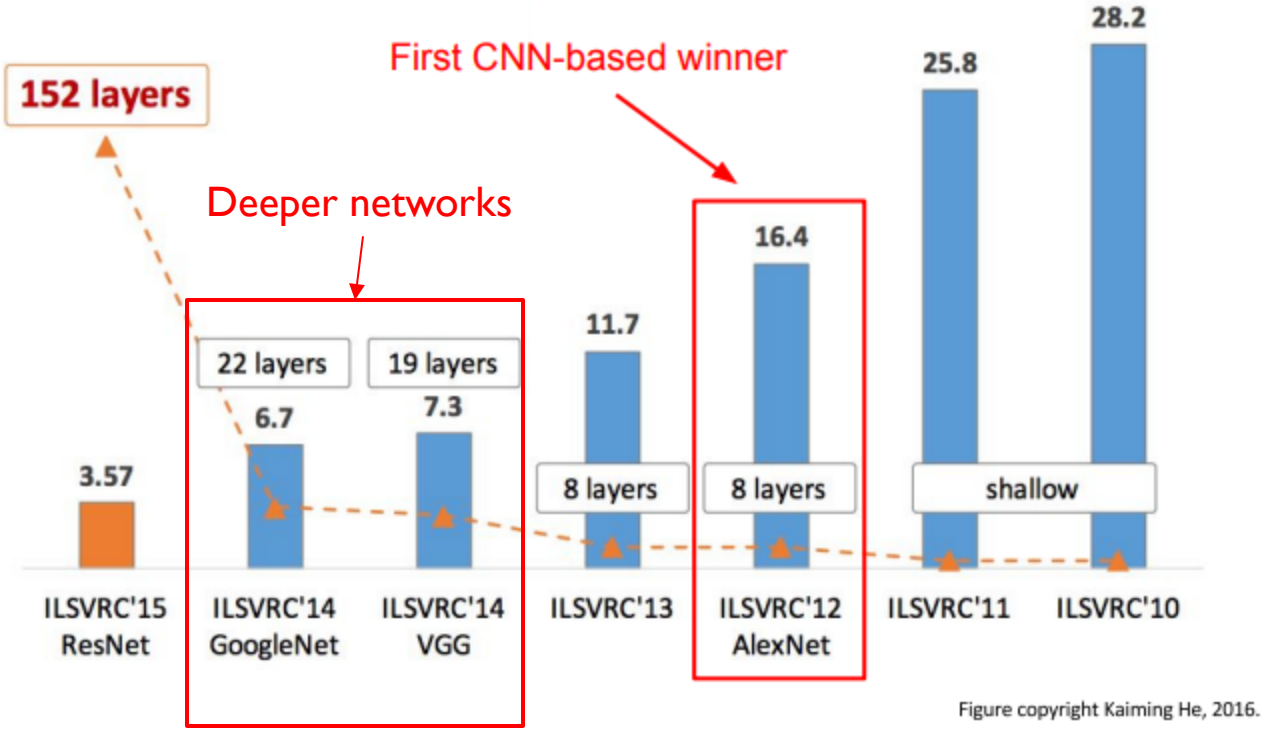


Figure copyright Kaiming He, 2016.

Stanford CS231n

CNN case studies

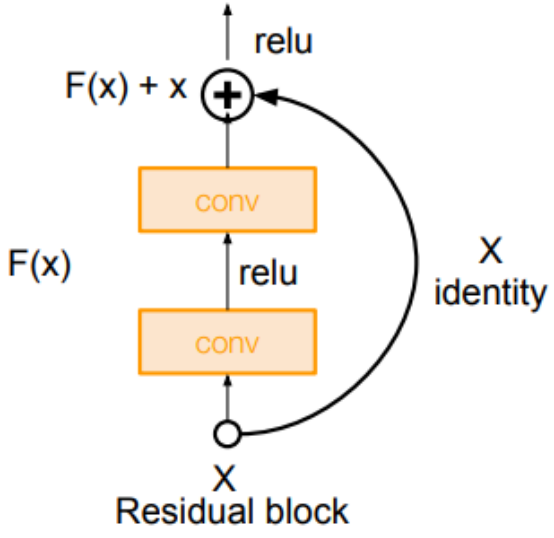
- ResNet 2015

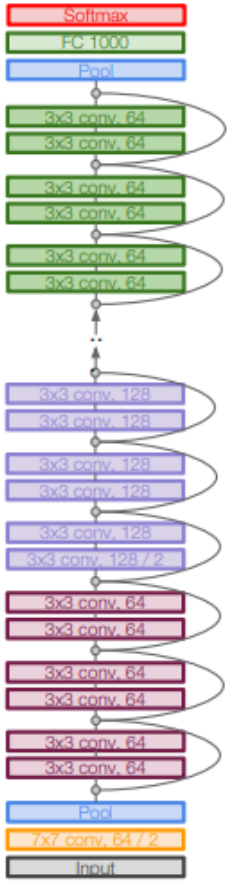
Very deep networks using residual connections.

- 152-layer model for ImageNet.
- Stack of residual blocks.

Intuition:

- Stacking more and more layers makes optimisation of the network very hard.
- A residual block enables information to “skip” or “shortcut” some connections, i.e., to be passed deeper in the network.
- Make optimisation of deeper networks easier.

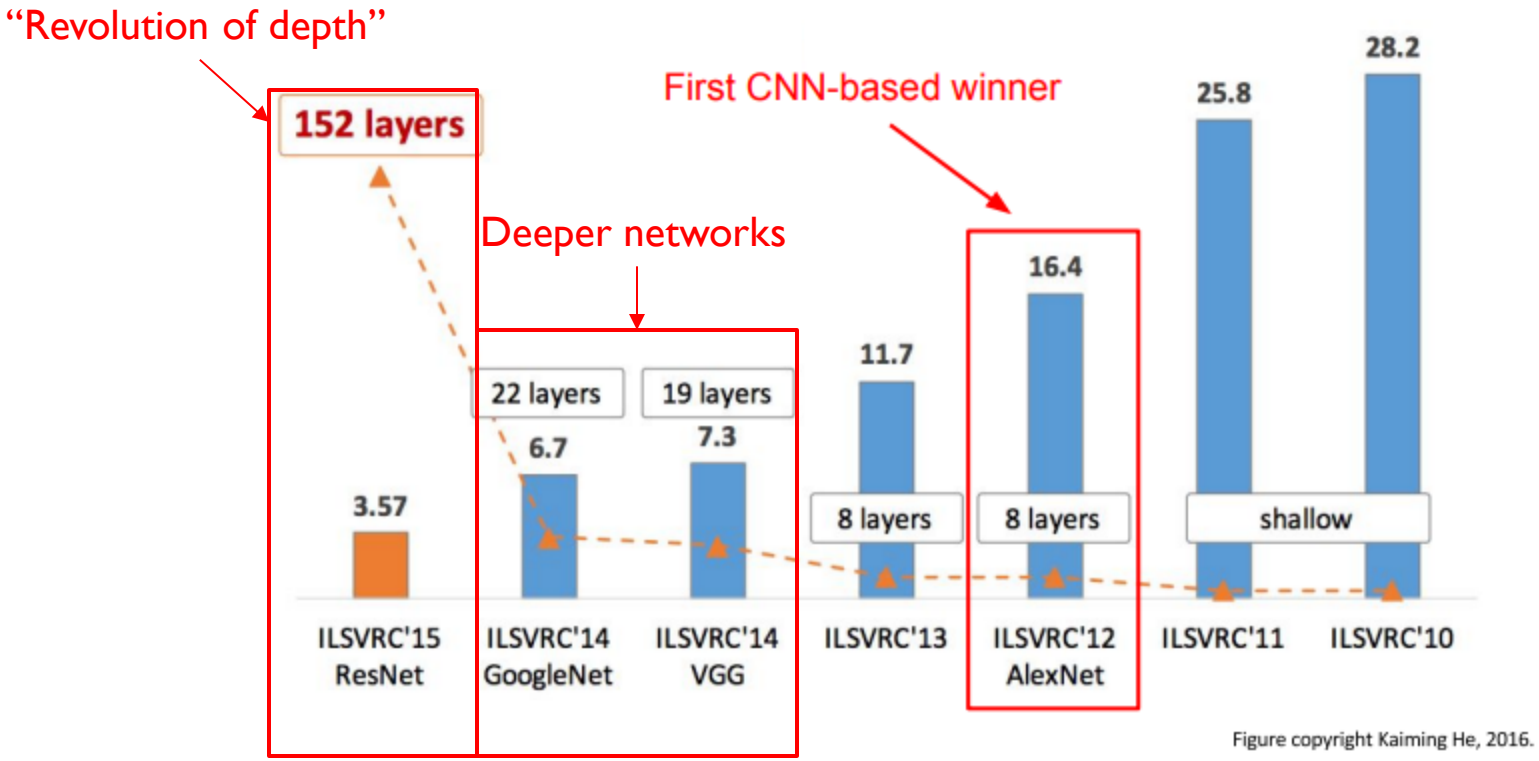




[He et al., 2015]
Stanford CS231n

CNN case studies

- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners.



Stanford CS231n

To go further

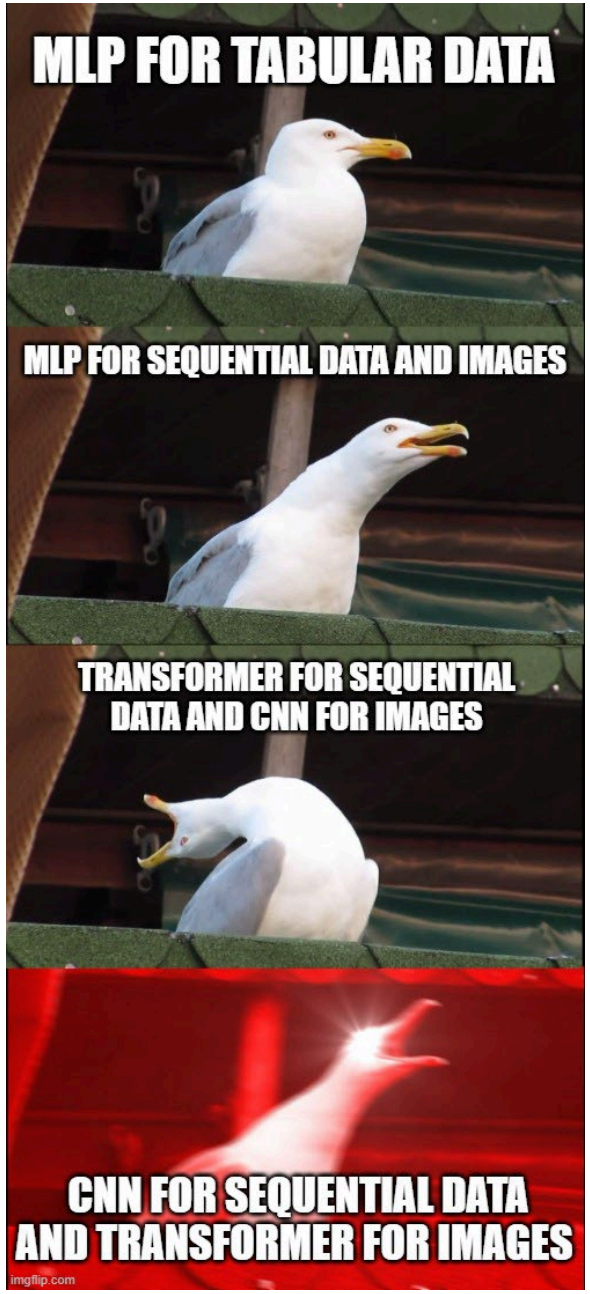
- Broader and deeper analysis of CNN architectures:

Stanford lecture on CNN Architectures on Youtube / Stanford slides

(e.g., performance vs complexity analysis, other architectures)

- LeNet-5: [Gradient-Based Learning Applied to Document Recognition, LeCun et al., 1998]
  http://yann.lecun.com/exdb/publis/pdf/lecun-98.pdf
- AlexNet: [ImageNet Classification with Deep Convolutional Neural Network, Krizhevsky et al., 2012]
  https://papers.nips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf
- VGGNet: [Very Deep Convolutional Networks for Large-scale Image Recognition, Simonyan and Zisserman et al., 2014]
  https://arxiv.org/pdf/1409.1556.pdf
- GoogLeNet: [Going deeper with convolutions, Szegedy et al., 2014]
  https://arxiv.org/pdf/1409.4842.pdf
- ResNet: [Deep Residual Learning for Image Recognition, He et al., 2015] https://arxiv.org/pdf/1512.03385.pdf

Wait, there is more!

- CNNs can also be used to deal with sequential data!
  - Univariate sequential data: 1D-convolution.
  - Multivariate sequential data: 2D-convolution.
- More on 1D-convolution: https://e2eml.school/convolution_one_d.html

Conclusion meme